Verlu User Manual

Verlu Team

2026-04-16

1 Verlu 应用平台 — 项目总览
1.1 1. 产品简介
1.2 2. 应用矩阵
1.3 3. 共享基础设施
1.3.1 3.1 统一认证
1.3.2 3.2 应用更新机制
1.3.3 3.3 扫码登录跨端授权
1.4 4. 技术规范
1.4.1 4.1 Android 公共技术栈
1.4.2 4.2 Cloud（KMP）技术栈
1.4.3 4.3 Admin Web 技术栈
1.5 5. 仓库结构说明
1.6 6. 模块间关联关系
2 快速开始指南
2.1 1. 环境要求
2.1.1 1.1 通用
2.1.2 1.2 Android 应用（app / talk / doctor / cnchess / music）
2.1.3 1.3 Cloud（KMP）
2.1.4 1.4 Supabase
2.2 2. 克隆仓库
2.3 3. Supabase 配置
2.3.1 3.1 Admin Web 环境变量配置
2.3.2 3.2 运行数据库迁移（可选，自建 Supabase）
2.3.3 3.3 部署 Edge Functions（可选，自建 Supabase）
2.4 4. Android 应用运行
2.4.1 4.1 使用 Android Studio
2.4.2 4.2 命令行构建
2.5 5. Cloud（KMP）运行
2.6 6. Admin Web 运行
2.7 7. 常见问题
2.7.1 Q：Gradle Sync 失败
2.7.2 Q：应用启动后直接显示认证页
2.7.3 Q：天气功能显示”获取失败”

- 2.7.4 Q：CnChess 无法匹配到对手
- 2.7.5 Q：Cloud Desktop 二维码登录无响应

3 整体架构设计
- 3.1 1. 系统全景
- 3.2 2. Android 应用内架构
  - 3.2.1 2.1 分层职责
  - 3.2.2 2.2 依赖注入
  - 3.2.3 2.3 认证状态管理
  - 3.2.4 2.4 导航
- 3.3 3. 数据同步策略
  - 3.3.1 3.1 本地优先（Room + Realtime）
  - 3.3.2 3.2 纯云端（CnChess 对局）
  - 3.3.3 3.3 代理转发（Doctor 本草 / Sync 天气）
- 3.4 4. Realtime 订阅架构
- 3.5 5. 安全设计
  - 3.5.1 5.1 行级安全（RLS）
  - 3.5.2 5.2 API 密钥隔离
  - 3.5.3 5.3 扫码登录安全机制
- 3.6 6. 应用更新机制

4 Sync 应用模块详解
- 4.1 1. 模块定位
- 4.2 2. 核心功能
- 4.3 3. 页面结构与路由
- 4.4 4. 数据架构
  - 4.4.1 4.1 本地数据库（Room）
  - 4.4.2 4.2 远程数据表
  - 4.4.3 4.3 天气数据流
- 4.5 5. 关键组件说明
  - 4.5.1 5.1 认证会话（AuthSessionViewModel）
  - 4.5.2 5.2 扫码授权（SSO）
  - 4.5.3 5.3 屏幕时长采集
  - 4.5.4 5.4 Realtime 订阅
- 4.6 6. 权限要求
- 4.7 7. 配置文件
- 4.8 8. 模块依赖

5 Doctor 应用模块详解
- 5.1 1. 模块定位
- 5.2 2. 核心功能
- 5.3 3. 页面结构与路由
- 5.4 4. 数据架构
  - 5.4.1 4.1 本地数据库（Room）
  - 5.4.2 4.2 网络请求路径
  - 5.4.3 4.3 离线策略
- 5.5 5. 关键组件说明
  - 5.5.1 5.1 Markdown 渲染
  - 5.5.2 5.2 搜索功能

- 5.5.3 5.3 收藏管理
- 5.6 6. 权限要求
- 5.7 7. Edge Function 说明（herb-proxy）
- 5.8 8. 配置文件
- 5.9 9. 模块依赖

6 Talk 应用模块详解
- 6.1 1. 模块定位
- 6.2 2. 核心功能
- 6.3 3. 页面结构与路由
- 6.4 4. 数据架构
  - 6.4.1 4.1 本地数据库（Room）
  - 6.4.2 4.2 远程数据表
  - 6.4.3 4.3 Realtime 订阅
- 6.5 5. 关键流程详解
  - 6.5.1 5.1 发送消息
  - 6.5.2 5.2 消息撤回（软删除）
  - 6.5.3 5.3 好友申请流程
  - 6.5.4 5.4 扫码加好友
  - 6.5.5 5.5 扫码授权登录（为 Cloud 桌面端服务）
- 6.6 6. 好友关系订阅（防断线重试）
- 6.7 7. 安全设计要点
- 6.8 8. 权限要求
- 6.9 9. 配置文件
- 6.10 10. 模块依赖

7 CnChess 应用模块详解
- 7.1 1. 模块定位
- 7.2 2. 核心功能
- 7.3 3. 页面结构与路由
- 7.4 4. 棋局数据架构
  - 7.4.1 4.1 核心数据模型
  - 7.4.2 4.2 云端数据表
- 7.5 5. 走子全流程
- 7.6 6. 棋规引擎（客户端）
  - 7.6.1 6.1 RuleEngine
  - 7.6.2 6.2 RepetitionJudge
  - 7.6.3 6.3 BoardCodec
- 7.7 7. 棋钟机制
- 7.8 8. 和棋提议流程
- 7.9 9. 回放模式
- 7.10 10. 单元测试
- 7.11 11. 权限要求
- 7.12 12. 配置文件
- 7.13 13. 模块依赖

8 Cloud 应用模块详解
- 8.1 1. 模块定位
- 8.2 2. 核心功能

- 8.3 3. 项目结构（KMP）
  - 8.3.1 3.1 共享层（commonMain）
- 8.4 4. 页面结构与导航（Voyager）
- 8.5 5. 存储架构
  - 8.5.1 5.1 用户文件隔离
  - 8.5.2 5.2 上传流程
  - 8.5.3 5.3 下载流程
- 8.6 6. Edge Function：cloud-files
- 8.7 7. 桌面端二维码登录
- 8.8 8. 深链协议（桌面端）
- 8.9 9. 应用更新
- 8.10 10. 配置文件
- 8.11 11. 构建命令
- 8.12 12. 模块依赖

9 Admin Web 模块详解
- 9.1 1. 模块定位
- 9.2 2. 核心功能
- 9.3 3. 支持的应用包名
- 9.4 4. 版本发布流程
  - 9.4.1 4.1 完整发布步骤
  - 9.4.2 4.2 版本状态管理
- 9.5 5. 权限控制
- 9.6 6. 环境配置
- 9.7 7. 部署方式
  - 9.7.1 7.1 本地开发
  - 9.7.2 7.2 生产部署（推荐）
  - 9.7.3 7.3 OAuth 登录配置（可选）
- 9.8 8. 技术细节
  - 9.8.1 8.1 单页应用结构
  - 9.8.2 8.2 实时订阅
- 9.9 9. 模块依赖

10 数据库与后端服务详解
- 10.1 1. 概述
- 10.2 2. 数据表详解
  - 10.2.1 2.1 用户与关系
  - 10.2.2 2.2 聊天
  - 10.2.3 2.3 设备指标（Sync 应用）
  - 10.2.4 2.4 象棋（CnChess）
  - 10.2.5 2.5 扫码登录
  - 10.2.6 2.6 应用版本
- 10.3 3. 关键 RPC 函数
  - 10.3.1 3.1 `make_move_v2`（象棋走子）
  - 10.3.2 3.2 `cnchess_sync_timeout_if_due`
  - 10.3.3 3.3 `get_my_room_ids`
- 10.4 4. Edge Functions
  - 10.4.1 4.1 `approve-login`

10.4.2 4.2 `weather-proxy`
10.4.3 4.3 `herb-proxy`
10.4.4 4.4 `cloud-files`
10.5 5. Storage Bucket
10.6 6. Realtime 订阅表清单
10.7 7. 迁移文件说明
11 跨模块核心流程
11.1 1. 统一账号注册与登录
11.2 2. 扫码授权登录（手机→桌面）
11.3 3. 好友系统（Talk ↔ CnChess 共享）
11.4 4. 象棋邀请与对局建立
11.5 5. 应用版本发布与更新
11.6 6. 天气数据流（Sync → 和风 API）
11.7 7. 中医内容访问（Doctor → zyapi）
11.8 8. 跨模块依赖总结

1 Verlu 应用平台 — 项目总览

> 版本：1.0.0 · 更新日期：2026-04-15

1.1 1. 产品简介

Verlu 是一套以**统一账号体系**为核心的多端应用平台，涵盖 Android 移动端、桌面端（JVM）与 Web 管理后台。所有应用共享同一个 **Supabase** 后端项目，通过统一认证、公共数据表与 Edge Function 协同工作，形成完整的”个人生活数字化”生态。

1.2 2. 应用矩阵

应用	模块路径	包名	平台	定位
Sync	`app/`	`cn.verlu.sync`	Android	设备状态监控与同步（电量、温度、屏幕时长、天气）
Talk	`talk/`	`cn.verlu.talk`	Android	即时通讯（单聊、

应用	模块路径	包名	平台	定位
				好友、扫码加友、扫码登录）
Doctor	`doctor/`	`cn.verlu.doctor`	Android	中医本草科普内容阅读（文章浏览、搜索、收藏）
CnChess	`cnchess/`	`cn.verlu.cnchess`	Android	在线中国象棋（好友对局、棋钟、判和、历史回放）
Music	`music/`	`cn.verlu.music`	Android	本地+在线音乐播放（发现、收藏、下载、睡眠定时）
Cloud	`cloud/`	`cn.verlu.cloud`	Android & Desktop	个人网盘（文件浏览、上传下载、跨端授权登录）
Admin Web	`admin-web/`	—	Web（浏览器）	运营管理后台（版本发布、灰度、强更）

1.3 3. 共享基础设施

```
┌──────────────────────────────────────────────────────────────────┐
│                          Supabase 项目                           │
│                                                                  │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐        │
│  │  Auth    │  │Postgrest │  │ Realtime │  │ Storage  │        │
│  │(统一账号)│  │(REST API)│  │(推送订阅)│  │(文件存储)│        │
```

```
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
│                                                          │
│  Edge Functions:                                         │
│  • approve-login   扫码登录批准                          │
│  • weather-proxy   和风天气代理                          │
│  • herb-proxy      本草 API 代理                         │
│  • cloud-files     S3 网盘操作                           │
└──────────────────────────────────────────────────────────┘

        ▲          ▲          ▲          ▲
        │          │          │          │
    Sync/Talk   Doctor     CnChess     Cloud
```

1.3.1 3.1 统一认证

所有应用均使用 **Supabase Auth（PKCE 流程）**，用邮箱密码注册/登录。用户信息存储于 `auth.users`，公开展示信息存储于 `profiles` 表。

1.3.2 3.2 应用更新机制

所有 Android 应用（及 Cloud 桌面端）均接入统一的**应用更新服务**：

1. 应用启动时查询 `app_releases` 表（按包名过滤）
2. 若后端标记为**强制更新**，应用展示不可跳过的更新弹窗
3. 若标记为**灰度更新**，仅特定用户可见
4. 管理员通过 Admin Web 上传安装包并配置版本信息

1.3.3 3.3 扫码登录跨端授权

Talk（手机端）与 Cloud（桌面端）共享 **`qr_login_sessions`** 机制：

1. 桌面 Cloud 生成二维码（含 `sessionId`）
2. 手机 Talk 扫码 → 调用 `approve-login` Edge Function
3. 桌面端通过 Realtime 订阅 session 状态变更，收到 `login_token` 后完成登录

1.4 4. 技术规范

1.4.1 4.1 Android 公共技术栈

技术	版本	用途
Kotlin	2.3.20	开发语言
Android Gradle Plugin	9.1.0	构建
Jetpack Compose BOM	2026.03.01	UI 框架
Material3	1.5.0-alpha16	设计系统

技术	版本	用途
Hilt	2.59.2	依赖注入
Navigation3	1.0.1	页面路由
Room	2.8.4	本地数据库
Supabase Kotlin SDK	3.5.0	后端接入
Ktor Client	3.4.2	HTTP 客户端
Coil3	3.4.0	图片加载
KSP	2.3.6	注解处理器

1.4.2 4.2 Cloud（KMP）技术栈

技术	版本	用途
Compose Multiplatform	1.10.3	跨端 UI
Voyager	—	导航
Koin	—	依赖注入
SQLDelight	—	跨端数据库
Supabase Kotlin SDK	3.1.4	后端接入

1.4.3 4.3 Admin Web 技术栈

技术	版本	用途
React	19	UI 框架
Vite	8	构建工具
TypeScript	6	开发语言
supabase-js	^2.103	后端接入

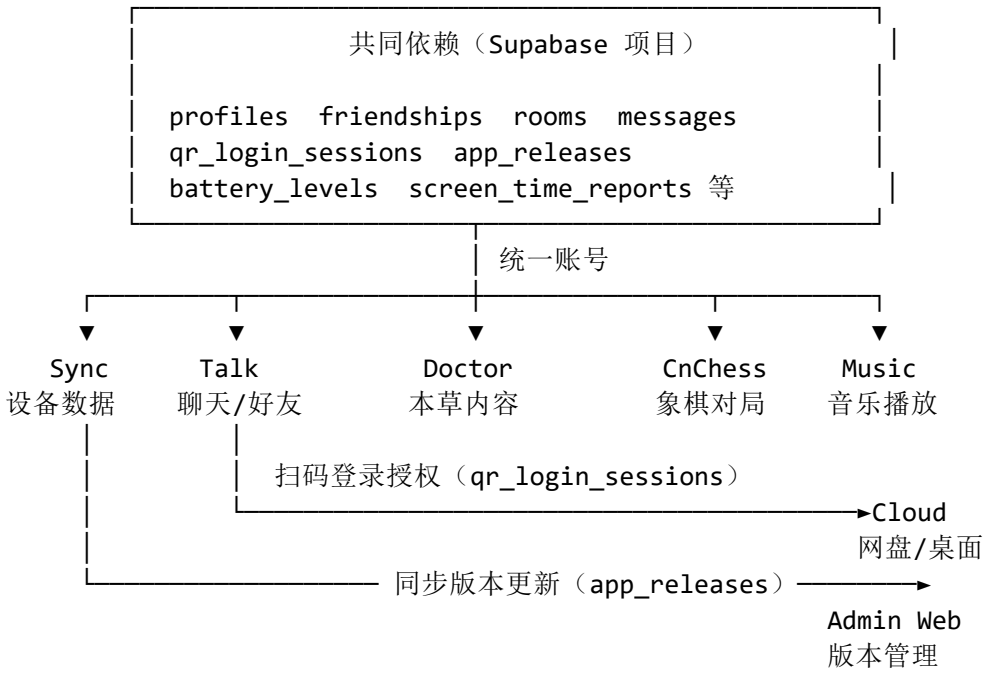
1.5 5. 仓库结构说明

```
sync/                          ← 根仓库（Gradle 多模块）
├── app/                       ← Sync Android 应用
├── talk/                      ← Talk Android 应用
├── doctor/                    ← Doctor Android 应用
├── cnchess/                   ← CnChess Android 应用
├── music/                     ← Music Android 应用
├── cloud/                     ← Cloud KMP 工程（独立 Gradle）
├── admin-web/                 ← Admin 后台（React/Vite）
├── supabase/
│   ├── migrations/            ← 数据库 Schema 迁移文件
│   └── functions/             ← Edge Functions（Deno/TypeScript）
```

```
├── docs/                    ← 本使用手册
└── gradle/
    └── libs.versions.toml   ← Android 统一依赖版本目录
```

> **注意**：`cloud/` 有独立的 `settings.gradle.kts` 和版本目录，与根工程的 Android 模块相互独立，需单独用 Android Studio 或 IntelliJ 打开。

1.6 6. 模块间关联关系



下一章：快速开始指南

2 快速开始指南

> 本章介绍如何搭建本地开发环境并运行各个子项目。

2.1 1. 环境要求

2.1.1 1.1 通用

工具	最低版本	说明
Git	2.40+	代码版本管理
JDK	17	Gradle 构建 & Android

工具	最低版本	说明
Android Studio	Meerkat（2024.3.1+）	Android 开发
Node.js	18+	Admin Web
npm	9+	Admin Web 包管理

2.1.2 1.2 Android 应用（app / talk / doctor / cnchess / music）

- Android SDK Platform：**API 35**
- Build-Tools：**35.x**
- 模拟器或真机：Android **API 26+**（cnchess 要求 **API 33+**）

2.1.3 1.3 Cloud（KMP）

- JDK 17（Desktop 端运行）
- IntelliJ IDEA 或 Android Studio（KMP 支持插件）
- 若打包 Desktop 端：对应平台工具链（Windows MSI / macOS dmg / Linux deb）

2.1.4 1.4 Supabase

- Supabase CLI（可选，用于本地迁移与 Edge Function 开发）
- 或直接使用已部署的 Supabase 云端项目

2.2 2. 克隆仓库

```
git clone <仓库地址>
cd sync
```

2.3 3. Supabase 配置

所有应用共用同一 Supabase 项目，配置已写入各模块 `SupabaseConfig.kt`（或 `.env`），**无需** 重复配置即可运行。

如需指向自己的 Supabase 项目，修改以下文件中的 `URL` 与 `ANON_KEY`：

模块	配置文件
Sync (app)	`app/src/main/java/cn/verlu/sync/data/remote/SupabaseConfig.kt`
Talk	`talk/src/main/java/cn/verlu/talk/data/remote/SupabaseConfig.kt`

模块	配置文件
Doctor	`doctor/src/main/java/cn/verlu/doctor/data/remote/SupabaseConfig.k`
CnChess	`cnchess/src/main/java/cn/verlu/cnchess/data/remote/SupabaseConfig`
Music	`music/src/main/java/cn/verlu/music/data/remote/SupabaseConfig.kt`
Cloud	`cloud/composeApp/src/commonMain/.../data/remote/SupabaseConfig.kt`
Admin Web	`admin-web/.env`（复制 `.env.example` 后填写）

2.3.1 3.1 Admin Web 环境变量配置

```
cd admin-web
cp .env.example .env
# 编辑 .env，填入 VITE_SUPABASE_URL 和 VITE_SUPABASE_ANON_KEY
```

2.3.2 3.2 运行数据库迁移（可选，自建 Supabase）

```
# 安装 Supabase CLI
npm install -g supabase

# 链接项目
supabase link --project-ref <你的项目ID>

# 执行所有迁移
supabase db push
```

2.3.3 3.3 部署 Edge Functions（可选，自建 Supabase）

```
# 部署全部 Functions
supabase functions deploy approve-login
supabase functions deploy weather-proxy
supabase functions deploy herb-proxy
supabase functions deploy cloud-files

# 设置 Secrets（以 weather-proxy 为例）
supabase secrets set QWEATHER_PROJECT_ID=xxx
supabase secrets set QWEATHER_KID=xxx
supabase secrets set QWEATHER_PRIVATE_KEY_B64=xxx
supabase secrets set QWEATHER_API_HOST=xxx

# herb-proxy
supabase secrets set HERB_API_KEY=xxx
supabase secrets set HERB_API_BASE=https://api.zyapi.com

# cloud-files
supabase secrets set S3_ENDPOINT=xxx
supabase secrets set S3_BUCKET=xxx
```

```
supabase secrets set S3_REGION=xxx
supabase secrets set S3_ACCESS_KEY=xxx
supabase secrets set S3_SECRET_KEY=xxx
supabase secrets set SUPABASE_SERVICE_ROLE_KEY=xxx
```

2.4 4. Android 应用运行

2.4.1 4.1 使用 Android Studio

1. 打开 Android Studio → **Open** → 选择 `sync/`（根目录）
2. 等待 Gradle Sync 完成（首次约 2–5 分钟）
3. 在顶部模块下拉选择目标模块（`:app`、`:talk`、`:doctor`、`:cnchess`、`:music`）
4. 选择设备/模拟器 → 点击 **Run**

2.4.2 4.2 命令行构建

```
# Debug APK（以 talk 为例）
./gradlew :talk:assembleDebug

# Release APK（需配置签名）
./gradlew :talk:assembleRelease

# 所有模块 Debug APK
./gradlew assembleDebug

# 单元测试
./gradlew :cnchess:testDebugUnitTest
```

构建产物位于各模块 `build/outputs/apk/` 目录。

2.5 5. Cloud（KMP）运行

`cloud/` 是**独立的 Gradle 工程**，需单独打开。

```
cd cloud

# Android
./gradlew :composeApp:assembleDebug

# Desktop JVM 运行
./gradlew :composeApp:run

# Desktop 打包（Windows）
./gradlew :composeApp:packageMsi
```

```
# Desktop 打包（macOS）
./gradlew :composeApp:packageDmg
```

2.6 6. Admin Web 运行

```
cd admin-web

# 安装依赖
npm install

# 开发服务器（默认 http://localhost:5173）
npm run dev

# 生产构建
npm run build

# 预览生产构建
npm run preview
```

2.7 7. 常见问题

2.7.1 Q：Gradle Sync 失败

- 确认 JDK 版本为 17（`java -version`）
- 检查 `gradle/wrapper/gradle-wrapper.properties` 中 Gradle 版本是否可下载
- 尝试：`./gradlew --stop && ./gradlew :app:assembleDebug`

2.7.2 Q：应用启动后直接显示认证页

- 正常现象：未登录时所有应用都会跳转到邮箱登录页
- 使用项目提供的测试账号或在 Supabase Dashboard 注册新用户

2.7.3 Q：天气功能显示”获取失败”

- 需要 Edge Function `weather-proxy` 已部署并配置了和风天气密钥
- 检查设备是否授予定位权限

2.7.4 Q：CnChess 无法匹配到对手

- 确保双方账号已互为好友（在 Talk 中添加）
- 确认两个账号均在 CnChess 主页（在线状态）

2.7.5 Q：Cloud Desktop 二维码登录无响应

- 确认手机端 Talk 已登录
- 检查 Edge Function `approve-login` 是否正常运行
- 查看 Supabase Dashboard → Edge Functions → 日志

下一章：整体架构设计

3 整体架构设计

3.1 1. 系统全景

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                 客户端层                                  │
│                                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐   │
│   │  Sync    │  │  Talk    │  │  Doctor  │  │ CnChess  │  │  Music   │   │
│   │ Android  │  │ Android  │  │ Android  │  │ Android  │  │ Android  │   │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘   │
│        │             │             │             │                        │
│        └─────────────┴─────────────┴─────────────┘                        │
│                      │  Supabase Kotlin SDK                               │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                    Cloud（Android + Desktop JVM）                 │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                    Admin Web（React SPA）                         │   │
│   └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │ HTTPS / WebSocket
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          Supabase 平台                                    │
│                                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────────┐  ┌──────────────┐        │
│   │  Auth    │  │PostgREST │  │   Realtime   │  │   Storage    │        │
│   │          │  │ REST API │  │  WebSocket   │  │ (S3 兼容)    │        │
│   └──────────┘  └──────────┘  └──────────────┘  └──────────────┘        │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                    PostgreSQL 数据库                              │   │
│   │  profiles · friendships · rooms · messages                        │   │
│   │  chess_games · chess_moves · chess_invites                        │   │
│   │  battery_levels · temperature_levels · screen_time_reports        │   │
│   │  weather_snapshots · qr_login_sessions · app_releases             │   │
```

```
│   └──────────────────────────────────────────────────────────┘   │
│                                                                    │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                    Edge Functions（Deno）                    │   │
│   │  approve-login  ·  weather-proxy  ·  herb-proxy  ·  cloud-files│  │
│   └──────────────────────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────────┘
```

3.2 2. Android 应用内架构

所有 Android 应用（Sync / Talk / Doctor / CnChess / Music）均采用统一的**分层架构**，遵循 Google 推荐的 MVVM + Repository 模式。

3.2.1 2.1 分层职责

```
┌──────────────────────────────────────────┐
│           Presentation 层                 │
│                                           │
│   Screen (Composable)                     │
│       ↕ State / Event                     │
│   ViewModel (Hilt + StateFlow)            │
└──────────────────────────────────────────┘
                  │
                  │ 调用
                  ▼
┌──────────────────────────────────────────┐
│              Domain 层                    │
│                                           │
│   Repository Interface                    │
│   Domain Model（纯 Kotlin 数据类）         │
│   业务规则（如 RuleEngine / RepetitionJudge）│
└──────────────────────────────────────────┘
                  │
                  │ 实现
                  ▼
┌──────────────────────────────────────────┐
│               Data 层                     │
│                                           │
│   RepositoryImpl                          │
│       ├── Local: Room DAO + Entity        │
│       │       ↕ 映射                      │
│       └── Remote: Supabase SDK / DTO      │
│               ├── Postgrest（CRUD）       │
│               ├── Realtime（推送订阅）     │
│               ├── Auth（会话管理）         │
│               └── Functions（边缘计算）    │
└──────────────────────────────────────────┘
```

3.2.2 2.2 依赖注入

使用 **Hilt**（Dagger2 封装），各模块的 `AppModule.kt` 负责：

- 提供 `SupabaseClient`（单例）

- 提供 Room Database 及各 DAO
- 绑定 Repository 接口到实现类
- 提供 `IoDispatcher`（协程调度器）

3.2.3 2.3 认证状态管理

每个应用都有 `AuthSessionViewModel`，负责：

```
Supabase Auth Flow
      │
      ▼
SessionStatus: Initializing → NotAuthenticated / Authenticated
      │
      ▼ (collectLatest + 500ms 冷启动防抖)
AuthSessionViewModel
      │
      ├── isInitializing = true  →  显示 Splash
      ├── isInitializing = false
      │       isAuthenticated = false  →  导航到 Auth
      │       isAuthenticated = true   →  导航到 Home
      └── 主动登出  →  立即导航到 Auth（不防抖）
```

关键设计：冷启动防抖逻辑集中在 ViewModel，Navigation 层不写 `delay` 补丁，确保”已登录用户冷启动直接进首页，不会闪一下 Auth 页”。

3.2.4 2.4 导航

使用 **Navigation3**（`androidx.navigation3`），路由定义为密封类或 `@Serializable` 数据类，所有路由集中在 `*NavApp.kt` 文件中管理。

3.3 3. 数据同步策略

3.3.1 3.1 本地优先（Room + Realtime）

适用于：Talk（消息、好友）、Sync（设备指标）

```
┌──────────────────────────────────────────────────────────┐
│  UI（Compose）                                            │
│    collectAsStateWithLifecycle(Room Flow)                 │
│    立即响应本地数据变更，无网络延迟                          │
└──────────────────────────────────────────────────────────┘
                          │ 观察 Room Flow
┌──────────────────────────▼───────────────────────────────┐
│  Room（本地 SQLite）                                      │
│    upsertAll() 写入 → Flow emit → UI 刷新                 │
└──────────────────────────────────────────────────────────┘
                          │ 两路写入
              ┌──────────────────────────────┐
```

```
        ▼                                  ▼
  Realtime 推送                        HTTP Refresh
  （subscribeToXxxChanges）              （refreshXxx）
  Supabase→通知→Repository→刷新→Room    主动拉取→写入Room
```

3.3.2 3.2 纯云端（CnChess 对局）

象棋对局数据不写 Room，直接使用 `StateFlow<ChessGame?>` 维护内存状态：

```
make_move_v2 RPC
      │
      ├── 乐观更新：_gameState.value = optimisticGame（立即刷新 UI）
      │
      ▼ 异步
Supabase RPC 响应
      ├── ok=true：refreshGame()（从服务端同步最终状态）
      ├── ok=false（违规/超时）：throw → UI 展示错误 + 回滚
      └── 网络异常：fallback 直接写库（兜底，不影响体验）
```

3.3.3 3.3 代理转发（Doctor 本草 / Sync 天气）

```
客户端 → Supabase Edge Function → 第三方 API
                ↑
          统一鉴权（验证 JWT）
          密钥不下发客户端
          CORS 处理
```

3.4 4. Realtime 订阅架构

```
Supabase Realtime Server（WebSocket）
         │
         │  postgres_changes
         ▼
RealtimeChannel（客户端订阅对象）
         │
         ├── INSERT → Repository.refreshXxx() → Room
         ├── UPDATE → Repository.refreshXxx() → Room
         └── DELETE → Repository.refreshXxx() → Room
```

防断线设计：各 Repository 中的 `subscribeToXxxChanges()` 采用：

1. 清理旧 channel，避免重复订阅
2. `keepXxxSubscription = true` 标记意图
3. 指数退避重试（1s → 2s → 4s → 8s → 16s → 32s），不轮询数据

3.5 5. 安全设计

3.5.1 5.1 行级安全（RLS）

所有业务表均启用 RLS，典型策略：

表	策略	规则摘要
`friendships`	see own	`auth.uid() = requester_id OR addressee_id`
`friendships`	send	`auth.uid() = requester_id`
`friendships`	accept/reject	`auth.uid() = addressee_id`
`messages`	insert	必须是接受好友关系的 room 成员
`chess_games`	服务端 RPC	`SECURITY DEFINER` 函数统一校验
`app_releases`	write	仅 `app_release_admins` 成员

3.5.2 5.2 API 密钥隔离

密钥	位置	说明
和风天气 JWT	Supabase Secrets	仅 `weather-proxy` 函数可读
本草 API Key	Supabase Secrets	仅 `herb-proxy` 函数可读
S3 凭证	Supabase Secrets	仅 `cloud-files` 函数可读
Supabase Anon Key	客户端代码	公开可读，受 RLS 约束
Supabase Service Role	Edge Functions	服务端专用，不下发客户端

3.5.3 5.3 扫码登录安全机制

- `qr_login_sessions` 记录 `expires_at`，过期失效
- `approve-login` 函数校验调用者 JWT 身份再审批
- 一次性 `login_token`，使用后状态变为 `approved`

3.6 6. 应用更新机制

```
Android 应用启动
      │
      ▼
AppUpdateGate（门控 Composable）
      │
      ▼
查询 app_releases（按 package_name + enabled=true 过滤）
      │
```

```
    ├── 无更新 / 灰度不命中 → 正常显示内容
    ├── 软更新（force=false）→ 可跳过弹窗
    └── 强制更新（force=true）→ 不可跳过，必须下载
                  │
                  ▼
            下载 APK / 安装包（Storage URL）
```

管理员通过 **Admin Web** 上传包并配置版本信息，全平台统一生效。

下一章：Sync 模块详解

4 Sync 应用模块详解

> 包名：`cn.verlu.sync` · 模块路径：`app/` · 平台：Android（API 26+）

4.1 1. 模块定位

Sync 是平台的”设备健康看板”应用。它持续采集当前手机的**电量**、**机身温度**、**屏幕使用时长**，并通过 Supabase 实时同步到云端，同时集成**天气查询**（基于实时定位）、**账号管理**与**扫码授权登录**（供 Cloud 桌面端或其它应用使用）。

4.2 2. 核心功能

功能	入口页面	说明
电量监控	`Home → 电量 Tab`	显示当前电量百分比、设备型号，上报至 `battery_levels`
屏幕时长	`Home → 屏幕时长 Tab`	汇总今日 / 近 7 天使用统计，上报至 `screen_time_reports`
温度监控	`Home → 温度 Tab`	读取设备温度传感器，上报至 `temperature_levels`
天气查询	`Home → 天气 Tab`	基于 GPS 定位查询当前天气与 3 日预报
账号管理	`Profile`	修改头像/昵称、修改密码、退出登录
扫码授权	`QrScan` / 深链	扫码批准其他端（Cloud 桌面）的登

功能	入口页面	说明
		录请求
应用更新	启动时自动	从 `app_releases` 检查新版本

4.3 3. 页面结构与路由

```
SyncNavApp
│
├── Splash（isInitializing=true 期间）
├── Auth 子流（未登录）
│   ├── AuthRoute（邮箱输入）
│   ├── AuthEmailRoute（注册/登录）
│   └── AuthPasswordRoute（密码输入）
│
└── Home（已登录）
    ├── HomeScreen
    │   ├── Tab 0：电量
    │   ├── Tab 1：温度
    │   ├── Tab 2：屏幕时长
    │   └── Tab 3：天气
    ├── ProfileScreen
    ├── QrScanScreen（扫码相机）
    └── SSOAuthorize(sessionId, returnPackage)
```

深链：`syncapp://login`（PKCE 回调）、`syncapp://authorize_sso?session_id=...`（扫码授权触发）

4.4 4. 数据架构

4.4.1 4.1 本地数据库（Room）

```
AppDatabase
├── BatteryLevelDao       → battery_levels
├── TemperatureLevelDao   → temperature_levels
├── ScreenTimeReportDao   → screen_time_reports
└── WeatherSnapshotDao    → weather_snapshots（本地缓存）
```

4.4.2 4.2 远程数据表

表名	主键	作用
`battery_levels`	`user_id`（text）	最新电量快照，upsert 更新
`temperature_levels`	`user_id`（uuid）	最新温度快照

表名	主键	作用
screen_time_reports	(user_id, period)	今日/近7日屏幕时长
weather_snapshots	user_id (text)	天气缓存，避免频繁调 API

4.4.3 4.3 天气数据流

```
设备 GPS 定位
      │
      ▼
QWeatherApiClient
      │  GET /functions/v1/weather-proxy?lat=xx&lon=xx
      ▼
Edge Function: weather-proxy
      │  EdDSA JWT 签名请求
      ▼
和风天气 API（v7/weather/now + 3d）
      │
      ▼
WeatherRepositoryImpl
      │  写入 Room + 同步到 weather_snapshots
      ▼
WeatherViewModel → UI
```

4.5 5. 关键组件说明

4.5.1 5.1 认证会话（AuthSessionViewModel）

- 订阅 `supabase.auth.sessionStatus`
- 冷启动防抖 500ms（等待 PKCE 会话恢复）
- `isInitializing=false` 后稳定输出 `isAuthenticated`

4.5.2 5.2 扫码授权（SSO）

用户在 Cloud 桌面或其它端生成二维码后，用 Sync 扫码触发以下流程：

```
1. CameraX 识别二维码中的 session_id
2. 深链 syncapp://authorize_sso?session_id=xxx 触发
3. SSOAuthorizeViewModel 调用 approve-login Edge Function
4. Edge Function 验证身份 → 更新 qr_login_sessions.status = approved
5. 对方端通过 Realtime 收到状态变更 → 完成登录
```

4.5.3 5.3 屏幕时长采集

- 通过系统 `UsageStatsManager` 获取 App 使用统计
- 需要用户授予**使用情况访问权限**（Settings → 应用使用情况）

- 按 `today` / `last_7_days` 两个 period 维度上报

4.5.4 5.4 Realtime 订阅

各数据 Repository 订阅对应表的变更，采用 Realtime + 指数退避重试策略（不轮询），详见整体架构 §4。

4.6 6. 权限要求

Android 权限	用途
`ACCESS_FINE_LOCATION`	GPS 定位（天气）
`PACKAGE_USAGE_STATS`	屏幕时长采集
`CAMERA`	扫码功能
`INTERNET`	网络请求
`FOREGROUND_SERVICE`	后台数据上报（如有）

4.7 7. 配置文件

```
// app/src/main/java/cn/verlu/sync/data/remote/SupabaseConfig.kt
object SupabaseConfig {
    const val URL = "https://jlzfvxxwzcpvtzdemcpm.supabase.co"
    const val ANON_KEY = "..."
}
```

4.8 8. 模块依赖

```
app (Sync)
├── Supabase: Auth + Postgrest + Functions + Realtime
├── Room: AppDatabase（4 个实体）
├── Hilt: AppModule（提供 SupabaseClient / DB / Repositories）
├── Navigation3: SyncNavApp
├── CameraX + ML Kit: 扫码
├── Play Services Location: 定位
└── （逻辑独立，不依赖其它 Android 模块）
```

下一章：Doctor 模块详解

5 Doctor 应用模块详解

> 包名：`cn.verlu.doctor` · 模块路径：`doctor/` · 平台：Android（API 26+）

5.1 1. 模块定位

Doctor 是面向中医爱好者的**本草/中医科普内容阅读**应用。用户可浏览本草文章、搜索药材与方剂、查看 Markdown 格式的详情页，并支持**收藏**与**离线缓存**。所有内容 API 请求均通过 Edge Function **`herb-proxy`** 转发，API 密钥不下发到客户端，保证安全性。

5.2 2. 核心功能

功能	入口页面	说明
首页内容	`Home`	展示推荐文章列表、分类导航
搜索	`HerbSearch`	关键词搜索药材/方剂，结果本地缓存
文章详情	`HerbArticle(path)`	Markdown 渲染，支持图片、代码块
收藏	`Favorites`	本地 Room 管理收藏条目
浏览历史	自动记录	Room 缓存最近浏览记录
账号管理	`Profile`	同其它应用，统一 Supabase 账号
应用更新	启动时自动	从 `app_releases` 检查新版本

5.3 3. 页面结构与路由

```
DoctorNavApp
│
├── Splash（isInitializing=true 期间）
├── Auth 子流（未登录）
│   ├── AuthRoute
│   ├── AuthEmailRoute
│   └── AuthPasswordRoute
│
└── Home（已登录）
    ├── HerbMainShell（主框架，含底部导航）
    │   ├── Tab：首页内容
    │   ├── Tab：搜索
    │   └── Tab：收藏
    ├── HerbArticle(path)
    └── ProfileScreen
```

5.4 4. 数据架构

5.4.1 4.1 本地数据库（Room）

```
HerbDatabase
├── ArticleDao        → 文章列表缓存
├── SearchDao         → 搜索历史 & 结果缓存
├── BrowseHistoryDao  → 最近浏览记录
└── FavoriteDao       → 收藏条目
```

5.4.2 4.2 网络请求路径

Doctor 没有独立的 Supabase 业务表，所有内容均来自第三方本草 API，通过 Edge Function 代理：

```
HerbRepository（客户端）
      │
      │  GET /functions/v1/herb-proxy?p=/herb/search?q=...
      │  Header: Authorization: Bearer <用户JWT>
      ▼
Edge Function: herb-proxy（服务端）
      │  验证 JWT 合法性
      │  读取 HERB_API_KEY、HERB_API_BASE（Secrets）
      ▼
第三方本草 API（zyapi 等）
      │
      ▼
响应 JSON → 客户端解析 → 写入 Room → UI 展示
```

设计优势： - API Key 仅存于 Supabase Secrets，客户端无法获取 - 请求需携带有效 JWT，防止未授权调用 - 服务端统一处理 CORS、超时、错误格式

5.4.3 4.3 离线策略

- 首次加载文章时写入 Room 缓存
- 下次访问时先展示 Room 数据，后台静默刷新
- 收藏与浏览历史仅本地存储，不同步云端

5.5 5. 关键组件说明

5.5.1 5.1 Markdown 渲染

使用 **multiplatform-markdown-renderer** 库渲染文章详情：

- 支持标题、段落、列表、代码块、表格
- 图片通过 **Coil3** 异步加载
- 中医术语、药材名称保持原有格式

5.5.2 5.2 搜索功能

```
用户输入关键词
      │
      ▼
SearchViewModel.search(query)
      │  防抖（300ms 去重）
      ▼
HerbRepository.searchHerbs(query)
      │  先查 Room 缓存
      │  无缓存或缓存过期 → 调 herb-proxy
      ▼
结果写入 Room → StateFlow → UI 更新
```

5.5.3 5.3 收藏管理

- 收藏操作直接写 Room，响应即时
- 首页/详情页均可收藏/取消收藏
- 收藏列表按时间倒序展示

5.6 6. 权限要求

Android 权限	用途
`INTERNET`	网络请求

无需其它特殊权限。

5.7 7. Edge Function 说明（herb-proxy）

路径：`supabase/functions/herb-proxy/index.ts`

功能： 1. 解析请求头中的 `Authorization: Bearer <JWT>` 2. 调用 Supabase Auth 验证 JWT 有效性 3. 取查询参数 `p`（目标 API 路径） 4. 使用 `HERB_API_KEY` 发起 GET 请求到 `HERB_API_BASE + p` 5. 透传响应体与状态码

配置的 Secrets：

Key	说明
`HERB_API_KEY`	第三方本草 API 授权 Key

Key	说明
`HERB_API_BASE`	API 基础 URL，默认 `https://api.zyapi.com`

5.8 8. 配置文件

```
// doctor/src/main/java/cn/verlu/doctor/data/remote/SupabaseConfig.kt
object SupabaseConfig {
    const val URL = "https://jlzfvxxwzcpvtzdemcpm.supabase.co"
    const val ANON_KEY = "..."
}
```

5.9 9. 模块依赖

```
doctor
├── Supabase：Auth + Functions（herb-proxy）+ Realtime（会话恢复）
├── Room：HerbDatabase（文章/搜索/历史/收藏）
├── Hilt：AppModule
├── Navigation3：DoctorNavApp
├── multiplatform-markdown-renderer：文章详情渲染
├── Coil3：图片加载
└── （逻辑独立，不依赖其它 Android 模块）
```

下一章：Talk 模块详解

6 Talk 应用模块详解

> 包名：`cn.verlu.talk` · 模块路径：`talk/` · 平台：Android（API 26+）

6.1 1. 模块定位

Talk 是平台的**即时通讯**应用，支持用户间单聊、好友管理（邮箱搜索/二维码）、消息实时推送，以及跨端**扫码授权登录**（手机批准桌面/其他设备的登录请求）。

Talk 是平台中功能最复杂的模块，与 Cloud 桌面端共享扫码登录基础设施，与 CnChess 共享好友关系表。

6.2 2. 核心功能

功能	入口	说明
会话列表	`Home → 会话 Tab`	显示所有聊天会话，含最新消息预览和未读状态
单聊	`ChatRoom(roomId)`	文字消息发送/接收，支持消息撤回（软删除）
好友列表	`Home → 联系人 Tab`	显示已接受好友，按首字母分组
新朋友	`联系人 → 新的朋友 Tab`	显示待处理的好友申请，支持同意/拒绝
添加好友	弹窗（邮箱搜索）	输入邮箱搜索用户并发送申请
扫码加好友	`QrScanFriend`	扫描对方二维码快速添加
个人资料	`Profile`	展示本人二维码（供对方扫码），修改昵称/头像
扫码授权登录	`QrScan`（通用扫码）	批准 Cloud 桌面端或其它设备的登录请求
应用更新	启动时自动	从 `app_releases` 检查新版本

6.3 3. 页面结构与路由

```
TalkNavApp
│
├── Splash（isInitializing=true 期间）
├── Auth 子流（未登录）
│   ├── AuthRoute（邮箱入口）
│   │   └── Sync 扫码授权弹窗（SyncAuthorizeLauncher）
│   ├── AuthEmailRoute（邮箱/密码）
│   └── AuthPasswordRoute（密码）
│
└── Home（已登录）
    ├── HomeScreen（含底部导航）
    │   ├── 会话列表（ConversationListScreen）
    │   └── 联系人（ContactsScreen）
    │       ├── Tab 0：好友列表
    │       └── Tab 1：新的朋友（待处理申请）
    ├── ChatRoomScreen(roomId)
    ├── ProfileScreen（含二维码弹窗 ProfileQrDialog）
    └── QrScanFriendScreen（扫码加好友）
```

6.4 4. 数据架构

6.4.1 4.1 本地数据库（Room）

```
TalkDatabase
├── MessageDao         → 消息缓存（message_id, room_id, content, ...）
├── ConversationDao    → 会话列表缓存（room_id, peer_info, last_message, ...）
└── FriendshipDao      → 好友关系缓存（accepted + pending）
```

6.4.2 4.2 远程数据表

表名	作用
`profiles`	用户信息（头像、昵称、邮箱）
`friendships`	好友关系（pending/accepted/blocked）
`rooms`	聊天房间
`room_members`	房间成员（每个好友对生成一个 room）
`messages`	消息记录（含软删除 `deleted_at`）
`message_reads`	已读状态
`qr_login_sessions`	扫码登录会话

6.4.3 4.3 Realtime 订阅

订阅目标	触发动作	处理逻辑
`friendships`（INSERT/UPDATE/DELETE）	好友状态变化	`refreshFriends()` → Room upsert
`messages`（INSERT）	收到新消息	`refreshRoomMessages()` → Room upsert
`rooms`（UPDATE）	会话最新消息变化	`refreshConversations()` → Room upsert
`qr_login_sessions`（UPDATE）	扫码会话状态	`AuthFormViewModel` 收到批准信号

6.5 5. 关键流程详解

6.5.1 5.1 发送消息

```
用户输入 → ChatRoomViewModel.sendMessage(content)
        │
        ├── 乐观 UI：立即在本地消息列表末尾追加（待发送状态）
```

```
      ▼
supabase.postgrest["messages"].insert(...)
      │  RLS 校验：sender_id = auth.uid()，room_id 需在已接受好友关系中
      ▼
插入成功 → Realtime 推送给对方 → 对方 Repository 刷新 → 对方 UI 更新
```

6.5.2 5.2 消息撤回（软删除）

```
长按消息 → 撤回选项 → 更新 deleted_at = now()
      │
      ▼
Realtime UPDATE 推送 → 双方 UI 显示"此消息已撤回"占位文字
```

6.5.3 5.3 好友申请流程

```
发起方：
  输入邮箱 → searchUser(email) → 找到用户
      │
      ▼
  sendFriendRequest(addresseeId)
      │  insert into friendships(requester_id, addressee_id,
status='pending')
      ▼
  数据写入数据库

接收方（当 Realtime 正常时，秒级感知；断线时重连后自动恢复）：
  friendship INSERT 事件 → refreshFriends() → Room upsert
      │
      ▼
  ContactsViewModel.observePendingRequests(userId) → 更新 "新的朋友(1)" 徽标
      │
      ▼
  用户点击"同意" → acceptFriendRequest(friendshipId)
      │  update friendships set status='accepted'
      │  触发器 handle_friendship_accepted：创建 room + room_members，设置
room_id
      ▼
  双方 Realtime UPDATE 感知 → refreshFriends() → 好友出现在列表
```

注意：`prevent_duplicate_friend_request` 触发器会阻止重复申请，并在反向申请存在时抛 `reverse_request_exists` 异常（UI 会提示”对方已向你发送了好友申请”）。

6.5.4 5.4 扫码加好友

```
ProfileScreen → 展示本人 QR（内含 userId）
      │
对方扫描 → QrScanFriendScreen
      │  解析 QR 中的 userId
      ▼
QrScanFriendViewModel.sendFriendRequest(userId)
```

```
      │  走同 5.3 的申请流程
      ▼
UI 提示发送成功
```

6.5.5 5.5 扫码授权登录（为 Cloud 桌面端服务）

```
Cloud 桌面展示二维码（含 sessionId）
      │
Talk 用户扫码 → 解析 sessionId
      │
AuthFormViewModel.approveLogin(sessionId)
      │  调用 approve-login Edge Function
      ▼
Edge Function 验证 JWT → 更新 qr_login_sessions
      │
Cloud 桌面 Realtime 订阅 → 收到 login_token → 完成登录
```

6.6 6. 好友关系订阅（防断线重试）

`FriendRepositoryImpl` 中实现了**指数退避 Realtime 重连**策略：

```
// 不轮询，只重连 Realtime 频道
private fun startRealtimeRetryLoop() {
    // 失败后：1s → 2s → 4s → 8s → 16s → 32s 重试
}
```

并已通过 `alter publication supabase_realtime add table public.friendships` 将该表纳入 Realtime 发布。

6.7 7. 安全设计要点

- **消息发送 RLS**：发送者必须和接收者处于”已接受好友关系”的同一 room 中
- **消息查看 RLS**：用户只能看到自己所在 room 的消息
- **好友申请 RLS**：只能以自己为 `requester_id` 发起申请
- **房间创建**：仅通过服务端触发器（`SECURITY DEFINER`）创建，不允许客户端直接 insert

6.8 8. 权限要求

Android 权限	用途
`INTERNET`	网络请求

Android 权限	用途
CAMERA	扫码（加好友 / 授权登录）

6.9 9. 配置文件

```
// talk/src/main/java/cn/verlu/talk/data/remote/SupabaseConfig.kt
object SupabaseConfig {
    const val URL = "https://jlzfvxxwzcpvtzdemcpm.supabase.co"
    const val ANON_KEY = "..."
    const val STORAGE_BUCKET = "cloud-user-files"
}
```

6.10 10. 模块依赖

```
talk
├── Supabase：Auth + Postgrest + Realtime + Storage（附件）+ Functions
├── Room：TalkDatabase（消息/会话/好友）
├── Hilt：AppModule
├── Navigation3：TalkNavApp
├── CameraX + ML Kit + ZXing：扫码
├── Coil3：头像图片加载
└── （逻辑独立，通过 Supabase 共享数据与 Cloud 协作）
```

下一章：CnChess 模块详解

7 CnChess 应用模块详解

> 包名：`cn.verlu.cnchess` · 模块路径：`cnchess/` · 平台：Android（API 33+）

7.1 1. 模块定位

CnChess 是平台的**在线中国象棋**对战应用。好友间可发起邀请、进行棋局、受棋钟约束实时走子，服务端通过 RPC 函数完成走子校验、棋钟扣时、超时判负和棋等规则裁决。还支持历史对局回放与复盘。

7.2 2. 核心功能

功能	说明
好友管理	显示已接受好友列表，共享 `friendships` 表（与 Talk 同库）
邀请系统	向好友发送对局邀请，Realtime 实时通知对方
在线状态	`cnchess_presence` 表记录用户在线/前台/心跳时间
棋盘对局	完整中国象棋走子、吃子、将军、将死、困毙判断
服务端棋钟	服务端 `make_move_v2` RPC 扣除思考时间，防客户端作弊
超时判负	`cnchess_sync_timeout_if_due` RPC 检查超时并写结果
禁手判断	长将/长捉 → 禁着提示；三次重复 → 判和
和棋提议	一方提议和棋，对方可接受/拒绝
认输	主动认输，服务端写 `resigned` 状态
历史对局	`GameHistoryScreen` 列表，含结果与时间
棋局回放	`Game(gameId, startInReplayMode=true)` 逐步复盘
应用更新	从 `app_releases` 检查新版本

7.3 3. 页面结构与路由

```
CnChessNavApp
│
├── Splash（isInitializing=true 期间）
├── Auth 子流（未登录）
│   ├── AuthRoute
│   ├── AuthEmailRoute
│   └── AuthPasswordRoute
│
└── Home（已登录）
    ├── HomeScreen（好友列表主页）
    │   └── IncomingInviteDialog（收到邀请时弹出）
    ├── ProfileScreen
    ├── GameHistoryScreen
    └── GameScreen(gameId, startInReplayMode)
```

后台监听：`CnChessNavApp` 在 `LaunchedEffect` 中持续运行： - `InviteListenerViewModel` → Realtime 监听 `chess_invites` - `ActiveGameViewModel.checkOnce()` → 检查是否有进行中对局（冷启动自动续局） - 每 30 秒 `inviteVm.heartbeat()` → 更新 `cnchess_presence` 在线时间戳

7.4 4. 棋局数据架构

CnChess **不使用 Room**，所有对局状态均以 `StateFlow<ChessGame?>` 形式存于内存，服务端为单一数据源。

7.4.1 4.1 核心数据模型

```
data class ChessGame(
    val id: String,
    val redUserId / blackUserId: String,
    val board: BoardState,       // 10×9 棋盘状态
    val turnSide: Side,          // 当前行棋方 Red/Black
    val redTimeMs / blackTimeMs: Long,  // 剩余棋钟
    val status: ChessGameStatus, // Active/Finished/Resigned/Draw/Timeout
    val moveHistory: List<MoveRecord>,
    ...
)
```

7.4.2 4.2 云端数据表

表名	说明
`cnchess_presence`	用户在线状态（user_id, status, last_heartbeat）
`chess_invites`	邀请记录（from/to userId, status, game_id）
`chess_games`	对局主表（FEN, 棋钟, 状态, 走子方, 和棋字段等）
`chess_moves`	每步走子记录（含 FEN 前后、棋钟、标签）

7.5 5. 走子全流程

```
用户点击落子目标格
      │
GameViewModel.onDropPiece(from, to)
      │
RuleEngine.tryApplyMove(board, side, from, to)
      │  ← 非法走子立即拒绝（不出网络请求）
      ▼
buildMoveAnnotation(nextBoard, move, ...)
      │  计算 isCheck / isChase / positionHash / judgeTag
      ▼
RepetitionJudge.decide(history, annotation)
      │  检查：长将→禁着 / 长捉（含重复局面）→禁着 / 三次重复→和棋
      │  ← 违规时立即抛异常，UI 显示提示
      ▼
乐观更新：_gameState.value = optimisticGame（棋盘立即响应）
      │
      ▼
```

```
make_move_v2 RPC（Supabase）
      │  服务端再次校验：move_no, turn_side, fen_before
      │  服务端扣除棋钟：spent_ms（不信任客户端时间）
      │  服务端写 chess_moves
      │  服务端更新 chess_games（状态/棋钟/FEN）
      ▼
RPC 响应
  ok=true  → refreshGame()（同步服务端最终状态）
  ok=false → 解析错误码 → 回滚 UI + 提示
  网络异常 → fallback 直接写库 + refreshGame()
```

7.6 6. 棋规引擎（客户端）

位于 `cnchess/src/main/java/cn/verlu/cnchess/domain/chess/`：

7.6.1 6.1 RuleEngine

方法	功能
`legalMovesFrom(board, side, from)`	返回合法着法列表（包含过滤自将）
`tryApplyMove(board, side, from, to)`	验证并应用走子，返回新棋盘状态
`isInCheck(board, side)`	判断指定方是否被将军
`hasAnyLegalMove(board, side)`	判断是否存在合法走法（困毙检测）
`evaluateResult(board, turnSide)`	评估对局结果（将死/困毙/继续）
`buildMoveAnnotation(...)`	构建走子注解（将军/捉/局面哈希）
`isChaseMove(board, move)`	判断是否为捉子走法（捉车/炮/马/将）

棋子走法规则：王/仕/象/马/车/炮/兵均完整实现，含蹩马腿、象眼、炮需翻山、兵过河横走、将帅照面等规则。

7.6.2 6.2 RepetitionJudge

```
fun decide(history, annotation): JudgeDecision
```

判定优先级（按序，先命中先返回）：

1. **长将禁着**：当前方连续 3 步均为将军 → `illegal_long_check`
2. **长捉禁着**：当前方连续 3 步均为捉子，**且**局面已达三次重复 → `illegal_long_chase`
3. **三次重复和棋**：相同局面（positionHash）出现 3 次 → `threefold_repetition`

> **关键设计**：长捉必须叠加”局面重复”条件，避免普通持续施压被误判为禁手。

7.6.3 6.3 BoardCodec

- `encode(BoardState): String` → FEN 格式字符串（服务端兼容）
- `decode(String): BoardState` → 从 FEN 恢复棋盘
- `initialBoard(): BoardState` → 标准开局布局

7.7 7. 棋钟机制

```
服务端存储：chess_games.red_time_ms / black_time_ms（剩余时间，毫秒）

客户端计算已用时间：
  spentMs = now() - lastMoveAt（或 createdAt）

服务端 make_move_v2 接收 p_spent_ms，扣除后更新棋钟
  nextRedTimeMs = redTimeMs - spentMs （若 side=red）

超时检测：
  cnchess_sync_timeout_if_due(game_id, user_id, current_time)
  → 服务端判断是否超时，写 status='timeout', winner=对方
```

7.8 8. 和棋提议流程

```
方A 点击"提议和棋"
      │
GameViewModel.offerDraw()
      │  update chess_games set draw_offer_by_user_id = uid
      ▼
方B Realtime 感知 draw_offer_by_user_id 不为空 → UI 弹出和棋提议弹窗
      │
      ├── 接受 → make_move_v2（p_draw_reason='mutual_agreement'）→
status='draw'
      └── 拒绝 → update draw_offer_by_user_id = null
```

7.9 9. 回放模式

```
GameScreen(gameId, startInReplayMode=true)
      │
ReplayControllerViewModel 加载 moveHistory
      │
逐步前进/后退走子（纯客户端计算，不发网络请求）
      │
GameScreen 同步渲染每步棋盘状态
```

7.10 10. 单元测试

位于 `cnchess/src/test/java/cn/verlu/cnchess/`：

测试类	覆盖内容
`RuleEngineTest`	合法走法、将军过滤、编解码往返
`RepetitionJudgeTest`	三次重复/长将/长捉判断、优先级
`ReplayControllerTest`	回放步进/回退逻辑

运行命令：

```
./gradlew :cnchess:testDebugUnitTest
```

7.11 11. 权限要求

Android 权限	用途
`INTERNET`	网络请求

最低 SDK：**API 33**（`minSdk = 33`）

7.12 12. 配置文件

```
// cnchess/src/main/java/cn/verlu/cnchess/data/remote/SupabaseConfig.kt
object SupabaseConfig {
    const val URL = "https://jlzfvxxwzcpvtzdemcpm.supabase.co"
    const val ANON_KEY = "..."
}
```

7.13 13. 模块依赖

```
cnchess
├── Supabase：Auth + Postgrest + Realtime（presence/invite/game）
├── Hilt：AppModule
├── Navigation3：CnChessNavApp
├── 自研 RuleEngine + RepetitionJudge + BoardCodec（domain/chess/）
├── （不使用 Room，所有状态在内存）
└── （通过 Supabase 共享 friendships 表与 Talk 好友数据）
```

下一章：Cloud 模块详解

8 Cloud 应用模块详解

> 包名：`cn.verlu.cloud` · 模块路径：`cloud/` · 平台：Android + Desktop JVM（Kotlin Multiplatform）

8.1 1. 模块定位

Cloud 是平台唯一的**跨端**应用，同时支持 Android 手机和桌面端（Windows/macOS/Linux）。其核心功能是**个人网盘**：文件浏览、上传、下载、分享，对象存储通过 Edge Function **`cloud-files`** 对接 S3 兼容存储（阿里云 OSS / Cloudflare R2 等）。

桌面端还支持**二维码登录**，需手机端 Talk 扫码批准，完成跨端身份传递。

> **注意**：`cloud/` 是独立的 Gradle 工程（Kotlin Multiplatform），需单独在 IDE 中打开，不属于根仓库的 Android 多模块项目。

8.2 2. 核心功能

功能	平台	说明
文件浏览	Android + Desktop	按用户隔离的目录树，前缀 `owners/{uid}/`
文件上传	Android + Desktop	预签名上传，大文件分片，小文件服务端 proxy
文件下载	Android + Desktop	预签名下载 URL，支持断点续传（通过传输队列）
文件删除	Android + Desktop	服务端 `cloud-files` delete action
文件移动/重命名	Android + Desktop	服务端 `cloud-files` move action
传输队列管理	Android + Desktop	SQLDelight 持久化传输任务状态
邮箱登录	Android + Desktop	与其它应用统一的 Supabase Auth
二维码登录	Desktop 专有	桌面展示 QR，手机 Talk 扫码批准
深链协议	Desktop 专有	`verlucloud://` 协议处理登录回调

功能	平台	说明
应用更新	Android + Desktop	`app_releases` 检查，Android 下载 APK，桌面下载安装包

8.3 3. 项目结构（KMP）

```
cloud/
├── composeApp/
│   ├── src/
│   │   ├── commonMain/         ← 跨端共享代码（UI + 业务逻辑）
│   │   ├── androidMain/        ← Android 平台特化
│   │   └── jvmMain/            ← Desktop JVM 平台特化
│   └── build.gradle.kts
├── gradle/
│   └── libs.versions.toml     ← Cloud 独立版本目录
└── settings.gradle.kts
```

8.3.1 3.1 共享层（commonMain）

绝大多数 UI 和业务逻辑写在 `commonMain`，平台差异通过 `expect/actual` 机制隔离：

类型	共享实现	平台特化
UI	Compose Multiplatform	底层渲染由平台处理
导航	Voyager	—
DI	Koin	—
数据库	SQLDelight（CloudDatabase）	SQLite 驱动由平台提供
存储适配	CloudEdgeFunctionAdapter	S3 操作通用
登录协议	Auth 流程	桌面额外支持 QR + 深链

8.4 4. 页面结构与导航（Voyager）

```
CloudAppRoot（App.kt）
      │
      ├── SessionLoadingScreen（会话加载中）
      │
      ├── AuthScreen（未登录）
      │   ├── Landing（欢迎页，桌面展示 QR）
      │   ├── EmailStep（输入邮箱）
      │   └── PasswordStep（输入密码）
      │
```

```
      └── ExplorerScreen（已登录，主界面）
          ├── 文件列表
          ├── 上传入口
          ├── 传输队列管理
          └── 个人信息 / 退出
```

8.5 5. 存储架构

8.5.1 5.1 用户文件隔离

所有文件对象的 S3 key 以 `owners/{userId}/` 为前缀，RLS 或 Edge Function 层均强制此约束，用户无法访问他人文件。

8.5.2 5.2 上传流程

```
用户选择文件
      │
CloudEdgeFunctionAdapter.upload(fileInfo)
      │
      ├── 小文件（< 阈值）: POST /cloud-files?action=upload_proxy
      │       → Edge Function 接收后直接 put 到 S3
      │
      └── 大文件：多步分片上传
              │  POST /cloud-files?action=create_multipart
              │  → 获取 uploadId
              ├── 分片循环：POST /cloud-files?action=upload_part
              └── 完成：POST /cloud-files?action=complete_multipart
      │
SqlDelightTransferRepository
      │  记录传输任务状态（pending/uploading/done/failed）
      ▼
传输完成 → 刷新文件列表
```

8.5.3 5.3 下载流程

```
用户点击下载
      │
POST /cloud-files?action=presign_download
      │  Edge Function 生成预签名 URL（时效性）
      ▼
客户端使用预签名 URL 直接从 S3 下载
      │
SqlDelightTransferRepository 记录进度
      ▼
下载完成 → 系统通知 / 本地文件可用
```

8.6 6. Edge Function：cloud-files

路径：`supabase/functions/cloud-files/index.ts`

所有 S3 操作均通过此函数代理，确保： - AWS 签名/密钥不下发客户端 - 所有操作必须携带有效 Supabase JWT - 文件路径强制 `owners/{uid}/` 前缀

action	说明
`list`	列出指定前缀下的文件和文件夹
`presign_upload`	生成预签名上传 URL
`presign_download`	生成预签名下载 URL
`create_multipart`	初始化分片上传
`upload_part`	上传单个分片
`complete_multipart`	完成分片上传
`upload_proxy`	小文件服务端代理上传
`delete`	删除文件/文件夹
`move`	移动/重命名（服务端 copy + delete）

配置的 Secrets：

Key	说明
`S3_ENDPOINT`	S3 兼容存储端点（如 oss-cn-hangzhou.aliyuncs.com）
`S3_BUCKET`	Bucket 名称
`S3_REGION`	地域
`S3_ACCESS_KEY`	访问密钥 ID
`S3_SECRET_KEY`	访问密钥 Secret
`SUPABASE_SERVICE_ROLE_KEY`	验证用户 JWT 所需

8.7 7. 桌面端二维码登录

```
Cloud Desktop 启动 / 未登录
      │
AuthScreen Landing 展示 QR（使用 qrose 库）
      │  QR 内容：sessionId（已写入 qr_login_sessions）
      │  订阅 qr_login_sessions 对应行的 Realtime 变更
      ▼
用户用 Talk 手机扫码
      │  approve-login Edge Function 被调用
      │  qr_login_sessions.status = 'approved'
      │  qr_login_sessions.login_token = <一次性 token>
```

```
      ▼
Cloud Desktop Realtime 收到 UPDATE 事件
      │  取出 login_token
      ▼
SupabaseAuthRepository.exchangeToken(login_token)
      │  完成会话建立
      ▼
导航到 ExplorerScreen（已登录）
```

8.8 8. 深链协议（桌面端）

桌面端注册 `verlucloud://` 自定义协议：

深链	触发场景
`verlucloud://login#access_token=...`	邮件链接登录回调
`verlucloud://reset-password#...`	密码重置回调

通过 `DesktopProtocolRegistrar` 在操作系统层注册协议处理器。

8.9 9. 应用更新

- **Android**：`CloudAndroidUpdateGate` → 查询 `app_releases`（`package_name = 'cn.verlu.cloud'`）
- **Desktop**：`CloudDesktopUpdateGate` → 查询（`package_name = 'cn.verlu.cloud.desktop'`），按平台提供 MSI/DMG/DEB 下载链接

8.10 10. 配置文件

```kotlin
// cloud/composeApp/src/commonMain/.../data/remote/SupabaseConfig.kt
object SupabaseConfig {
    const val URL = "https://jlzfvxxwzcpvtzdemcpm.supabase.co"
    const val ANON_KEY = "..."
    const val STORAGE_BUCKET = "cloud-user-files"
    const val REDIRECT_URI = "verlucloud://login"
    const val DESKTOP_QR_SESSION_EXPIRY_SECONDS = 300
}
```

8.11 11. 构建命令

```
cd cloud

# Android Debug APK
./gradlew :composeApp:assembleDebug

# Desktop 运行（开发用）
./gradlew :composeApp:run

# Desktop Windows 打包
./gradlew :composeApp:packageMsi

# Desktop macOS 打包
./gradlew :composeApp:packageDmg

# Desktop Linux 打包
./gradlew :composeApp:packageDeb
```

8.12 12. 模块依赖

```
cloud (KMP)
├── Supabase KMP SDK 3.1.4：Auth + Postgrest + Realtime + Storage + Functions
├── SQLDelight：跨端传输队列持久化（CloudDatabase）
├── Voyager：KMP 导航
├── Koin：跨端依赖注入
├── qrose：二维码生成（桌面 QR 登录）
├── Compose Multiplatform 1.10.3：跨端 UI
└── （通过 qr_login_sessions + approve-login 与 Talk 协作完成扫码登录）
```

下一章：Admin Web 模块详解

9 Admin Web 模块详解

> 路径：`admin-web/` · 平台：Web 浏览器（SPA）· 技术栈：React 19 + Vite 8 + TypeScript 6

9.1 1. 模块定位

Admin Web 是平台的**运营管理后台**，专供管理员使用。核心职责是管理所有应用（Android APK + Desktop 安装包）的**版本发布**：上传安装包、配置版本信息（强更/灰度/更新日志）、上线/下线版本。

普通用户无需使用此后台，所有版本变更实时影响对应应用的启动时更新检查。

9.2 2. 核心功能

功能	说明
管理员登录	邮箱密码 + Google/GitHub OAuth
查看版本列表	按应用分类展示所有版本，含状态、创建时间
发布新版本	上传安装包（最大 200MB）、填写版本信息
启用/停用版本	切换 `enabled` 状态，立即影响客户端检查结果
编辑版本信息	修改版本名、更新日志、灰度配置等
删除版本	同时删除 Storage 中的安装包对象（best-effort）
权限提示	未被加入 `app_release_admins` 时，展示引导说明

9.3 3. 支持的应用包名

Admin Web 内置以下包名供选择：

包名	应用	平台
`cn.verlu.sync`	Sync	Android
`cn.verlu.talk`	Talk	Android
`cn.verlu.music`	Music	Android
`cn.verlu.doctor`	Doctor	Android
`cn.verlu.cnchess`	CnChess	Android
`cn.verlu.cloud`	Cloud	Android
`cn.verlu.cloud.desktop`	Cloud Desktop	Windows/macOS/Linux

支持的安装包格式：`.apk`、`.msi`、`.dmg`、`.deb`、`.exe`、`.zip`

9.4 4. 版本发布流程

9.4.1 4.1 完整发布步骤

```
1. 登录 Admin Web（管理员账号）
      |
2. 点击「发布新版本」
      |
```

```
3. 填写表单：
   ├── 包名（选择目标应用）
   ├── Version Code（整数，递增）
   ├── Version Name（如 1.2.3）
   ├── 更新日志（Changelog）
   ├── 最低支持版本（低于此版本强制更新）
   ├── 是否强制更新（force = true/false）
   ├── 是否灰度（仅特定用户可见）
   └── 上传安装包文件（最大 200MB）
       │
4. 提交 → 文件上传到 Storage bucket: app-releases
       │    路径：{packageName}/{versionCode}/{filename}
       │
5. 记录写入 app_releases 表
       │    含 download_url（Storage 公开 URL）
       │
6. 客户端下次启动时检测到新版本
```

9.4.2 4.2 版本状态管理

```
新发布 → enabled=true（默认上线）
       │
管理员可随时切换：
       ├── 停用（enabled=false）→ 客户端不再检测到此版本
       └── 重新启用（enabled=true）→ 恢复可见

自动规则：
       当一个包名新启用版本时，之前的 enabled=true 版本自动停用
       （由数据库触发器实现，见迁移文件）
```

9.5 5. 权限控制

Admin Web 依赖 **`app_release_admins`** 表实现权限控制：

```sql
-- 只有 app_release_admins 中的用户才能写入 app_releases
CREATE POLICY "admins_can_manage_releases"
ON public.app_releases
FOR ALL
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.app_release_admins
    WHERE user_id = auth.uid()
  )
)
```

添加管理员步骤： 1. 在 Supabase Dashboard → SQL Editor 执行：

```sql
INSERT INTO public.app_release_admins (user_id)
VALUES ('<目标用户的 auth.users id>');
```

2. 或通过 Supabase Dashboard → Table Editor → `app_release_admins` 添加记录。

> Admin Web 界面会在操作失败时提示"权限不足，请联系管理员将您加入 `app_release_admins` "。

9.6 6. 环境配置

```
# admin-web/.env（复制自 .env.example）
VITE_SUPABASE_URL=https://jlzfvxxwzcpvtzdemcpm.supabase.co
VITE_SUPABASE_ANON_KEY=your_anon_key_here
```

Storage 配置： - Bucket 名称：`app-releases` - 文件大小上限：**200MB**（由 Storage 策略限制） - 文件访问：公开可读（客户端直接下载）

9.7 7. 部署方式

9.7.1 7.1 本地开发

```
cd admin-web
npm install
npm run dev
# 访问 http://localhost:5173
```

9.7.2 7.2 生产部署（推荐）

Vercel 一键部署（推荐，免费额度足够）： 1. 在 Vercel 导入 `admin-web/` 目录 2. 设置环境变量：`VITE_SUPABASE_URL`、`VITE_SUPABASE_ANON_KEY` 3. 构建命令：`npm run build`，输出目录：`dist`

Nginx 静态托管：

```
npm run build
# 将 dist/ 内容部署到 Web 服务器
```

9.7.3 7.3 OAuth 登录配置（可选）

若要启用 Google/GitHub OAuth，在 Supabase Dashboard → Authentication → Providers： 1. 启用 Google OAuth，配置 Client ID & Secret 2. 启用 GitHub OAuth，配置 Client ID & Secret 3. 在 Admin Web 设置的 Site URL 中添加回调 URL

9.8 8. 技术细节

9.8.1 8.1 单页应用结构

Admin Web 是简洁的**单文件 App**（`src/App.tsx`），无路由拆分：

```
App.tsx
├── 未登录状态 → 登录表单（邮箱/密码 + OAuth 按钮）
└── 已登录状态 → 版本管理界面
    ├── 版本列表（Table）
    ├── 发布新版本（Form + FileUpload）
    ├── 编辑版本（Modal）
    └── 管理员操作（启停/删除）
```

9.8.2 8.2 实时订阅

版本列表通过 **Supabase Realtime** 订阅 `app_releases` 变更，多个管理员同时操作时界面自动同步。

9.9 9. 模块依赖

```
admin-web
├── @supabase/supabase-js ^2.103：Auth + Postgrest + Storage + Realtime
├── React 19 + TypeScript 6：UI 框架
├── Vite 8：构建工具
└── （无路由库，单页应用）
```

下一章：数据库与后端服务

10 数据库与后端服务详解

> Supabase 项目 URL：`https://jlzfvxxwzcpvtzdemcpm.supabase.co`

10.1 1. 概述

平台所有应用共用**一个 Supabase 项目**，包含：

- **PostgreSQL 数据库**：业务数据存储
- **Auth**：统一用户认证
- **PostgREST**：自动生成的 REST API
- **Realtime**：基于 PostgreSQL logical replication 的实时推送

- **Storage**：文件存储（安装包、用户文件）
- **Edge Functions**：Deno 运行时，代理第三方服务

迁移文件位于 `supabase/migrations/`，按时间戳命名，记录所有 Schema 变更历史。

10.2 2. 数据表详解

10.2.1 2.1 用户与关系

10.2.1.1 `profiles`（用户公开信息）

字段	类型	说明
`id`	uuid PK	关联 `auth.users.id`
`display_name`	text	显示名称，默认 `'User'`
`avatar_url`	text	头像 URL
`email`	text	邮箱（冗余，方便查询）
`online_at`	timestamptz	最近在线时间
`created_at`	timestamptz	注册时间

触发器：`on_auth_user_created`（`AFTER INSERT ON auth.users`）→ 自动创建 `profiles` 记录，`display_name` 从 `raw_user_meta_data` 取 `full_name` / `user_name` / 邮箱前缀。

10.2.1.2 `friendships`（好友关系）

字段	类型	说明
`id`	uuid PK	关系记录 ID
`requester_id`	uuid FK→auth.users	发起方
`addressee_id`	uuid FK→auth.users	接收方
`status`	text	`pending` / `accepted` / `blocked`
`created_at`	timestamptz	申请时间
`updated_at`	timestamptz	状态变更时间
`room_id`	uuid FK→rooms	接受后绑定聊天室（可空）

唯一约束：`(requester_id, addressee_id)`，防止重复申请。

触发器： - `prevent_duplicate_friend_request`（BEFORE INSERT）：若存在 `(addressee_id→requester_id)` 的反向记录，抛 `reverse_request_exists` - `on_friendship_accepted`（BEFORE UPDATE）：`status pending→accepted` 时，自动创建 room + room_members，设置 `room_id`；同时删除反向 pending 请求

RLS 策略：

策略名	操作	条件
see own friendships	SELECT	`uid = requester_id OR addressee_id`
send friend request	INSERT	`uid = requester_id`
accept/reject request	UPDATE	`uid = addressee_id`
delete friendship	DELETE	`uid = requester_id OR addressee_id`

Realtime：已加入 `supabase_realtime` publication（迁移 `20260415234000`）。

10.2.2 2.2 聊天

10.2.2.1 `rooms`（聊天室）

字段	类型	说明
`id`	uuid PK	房间 ID

> 由 `handle_friendship_accepted` 触发器创建，无需客户端手动操作。

RLS：用户只能查看自己通过 `room_members` 关联的 room。

10.2.2.2 `room_members`（房间成员）

字段	类型	说明
`room_id`	uuid FK→rooms	房间
`user_id`	uuid FK→auth.users	成员

RLS：用户只能查看自己的成员记录。

10.2.2.3 `messages`（消息）

字段	类型	说明
`id`	uuid PK	消息 ID
`room_id`	uuid FK→rooms	所属房间
`sender_id`	uuid FK→auth.users	发送者

字段	类型	说明
`content`	text	消息内容
`type`	text	消息类型（默认 `text`）
`created_at`	timestamptz	发送时间
`deleted_at`	timestamptz	撤回时间（软删除，可空）

RLS：- INSERT：`sender_id = uid` 且 `room_id` 在用户已接受好友关系中 - SELECT/UPDATE：用户在该 room 的 `room_members` 中

10.2.2.4 `message_reads`（消息已读）

字段	类型	说明
`message_id`	uuid FK→messages	消息
`user_id`	uuid FK→auth.users	已读用户
`read_at`	timestamptz	已读时间

10.2.3 2.3 设备指标（Sync 应用）

10.2.3.1 `battery_levels`

字段	类型	说明
`user_id`	text PK	用户 ID（text 类型）
`battery_percent`	integer	电量百分比
`updated_at`	bigint	更新时间戳（毫秒）
`device_model`	text	设备型号
`device_friendly_name`	text	设备别名

RLS：authenticated 用户可读写所有行（多设备展示场景）。

10.2.3.2 `temperature_levels`

字段	类型	说明
`user_id`	uuid FK→auth.users PK	用户 ID
`temperature`	float	温度值
`updated_at`	timestamptz	更新时间

10.2.3.3 `screen_time_reports`

字段	类型	说明
`user_id`	uuid FK→auth.users	用户 ID
`period`	text	`today` / `last_7_days`
`data`	jsonb	各应用时长数据
`updated_at`	timestamptz	更新时间

约束：`period` 仅允许 `today` 或 `last_7_days`。复合主键 `(user_id, period)`。

10.2.3.4 `weather_snapshots`

字段	类型	说明
`user_id`	text PK	用户 ID（text）
`lat` / `lon`	float	定位坐标
`weather_json`	jsonb	和风天气响应缓存
`updated_at`	timestamptz	更新时间

10.2.4 2.4 象棋（CnChess）

10.2.4.1 `cnchess_presence`（在线状态）

字段	类型	说明
`user_id`	uuid PK FK→auth.users	用户
`status`	text	`online` / `away` / `offline`
`last_heartbeat`	timestamptz	最近心跳时间

RLS SELECT：自己的记录，或已接受好友的记录可见。

10.2.4.2 `chess_games`（对局）

字段	类型	说明
`id`	uuid PK	对局 ID
`red_user_id` / `black_user_id`	uuid FK	红黑双方用户

字段	类型	说明
`status`	text	`active` / `finished` / `aborted` / `timeout` / `resigned` / `draw`
`fen`	text	当前棋盘 FEN 字符串
`turn_side`	text	`red` / `black`
`move_no`	bigint	当前走子序号（乐观锁）
`red_time_ms` / `black_time_ms`	bigint	剩余棋钟（毫秒）
`winner_user_id`	uuid	胜者（可空）
`last_move_at`	timestamptz	最近走子时间
`draw_reason`	text	和棋原因
`draw_offer_by_user_id`	uuid	提议和棋的用户（可空）
`draw_offer_at`	timestamptz	提议时间
`created_at`	timestamptz	对局创建时间

Realtime：已加入 `supabase_realtime` publication。

10.2.4.3 `chess_invites`（邀请）

字段	类型	说明
`id`	uuid PK	邀请 ID
`from_user_id` / `to_user_id`	uuid FK	邀请双方
`status`	text	`pending` / `accepted` / `rejected` / `expired`
`game_id`	uuid FK→chess_games	接受后绑定对局（可空）
`created_at`	timestamptz	邀请时间

Realtime：已加入 `supabase_realtime` publication。

10.2.4.4 `chess_moves`（走子记录）

字段	类型	说明
`id`	uuid PK	走子记录 ID
`game_id`	uuid FK→chess_games	对局

字段	类型	说明
	CASCADE	
`move_no`	bigint	走子序号（唯一约束 with game_id）
`side`	text	`red` / `black`
`from_row` / `from_col` / `to_row` / `to_col`	integer	坐标
`fen_before` / `fen_after`	text	走子前后 FEN
`spent_ms`	bigint	本步耗时
`position_hash`	text	局面哈希（用于重复判断）
`is_check`	boolean	是否将军
`is_chase`	boolean	是否捉子
`judge_tag`	text	棋规标签（check/chase/illegal_long_check 等）
`created_at`	timestamptz	走子时间

Realtime：已加入 `supabase_realtime` publication。

10.2.5 2.5 扫码登录

10.2.5.1 `qr_login_sessions`

字段	类型	说明
`session_id`	uuid PK	会话 ID（QR 内容）
`status`	text	`pending` / `approved` / `expired`
`login_token`	text	一次性登录令牌（批准后生成）
`expires_at`	timestamptz	过期时间
`created_at`	timestamptz	创建时间

Realtime：已加入 `supabase_realtime` publication（Talk + Cloud 共用）。

RLS：任何人可 INSERT 和 SELECT（二维码展示与扫描均需）。

10.2.6 2.6 应用版本

10.2.6.1 `app_release_admins`（管理员）

字段	类型	说明
`user_id`	uuid PK FK→auth.users	管理员用户

10.2.6.2 `app_releases`（版本发布记录）

字段	类型	说明
`id`	uuid PK	版本记录 ID
`package_name`	text	应用包名
`version_code`	integer	版本号（整数，递增）
`version_name`	text	版本名（如 1.2.3）
`download_url`	text	安装包下载 URL
`file_size`	bigint	文件大小（字节）
`changelog`	text	更新日志
`min_supported_version`	integer	低于此版本强制更新
`force_update`	boolean	是否强制更新
`is_rollout`	boolean	是否灰度
`enabled`	boolean	是否上线
`created_by`	uuid FK→auth.users	发布者
`created_at`	timestamptz	发布时间

RLS：读取公开；写入需在 `app_release_admins` 中。

10.3 3. 关键 RPC 函数

10.3.1 3.1 `make_move_v2`（象棋走子）

```
make_move_v2(
  p_game_id, p_move_no, p_side,
  p_from_row, p_from_col, p_to_row, p_to_col,
  p_fen_before, p_fen_after,
  p_spent_ms, p_position_hash,
  p_is_check, p_is_chase, p_judge_tag, p_draw_reason
) RETURNS TABLE(ok boolean, new_move_no bigint, error text)
```

服务端校验逻辑： 1. 验证 `game_id` 对局为 active 且轮到 `p_side` 走子 2. 验证 `move_no` 匹配（乐观锁，防并发冲突） 3. 若 `judge_tag = 'illegal_long_check'` → 返回 `ok=false` 4. 若 `judge_tag = 'illegal_long_chase'` → 返回 `ok=false` 5. 检查棋钟超时（服务端权威时间） 6. 写入 `chess_moves`，更新 `chess_games`（FEN/棋钟/状态） 7. 若 `draw_reason` 非空 → `status='draw'`

10.3.2 3.2 `cnchess_sync_timeout_if_due`

检查对局是否已超时，若是则写 `status='timeout'`，`winner=对方`。客户端在页面恢复前台时调用。

10.3.3 3.3 `get_my_room_ids`

```
-- SECURITY DEFINER，返回当前用户的所有 room_id
SELECT room_id FROM room_members WHERE user_id = auth.uid();
```

供 Realtime 订阅过滤使用。

10.4 4. Edge Functions

10.4.1 4.1 `approve-login`

触发：Talk / Sync 手机端调用（用于批准 Cloud 桌面登录）

```
POST /functions/v1/approve-login
Authorization: Bearer <用户JWT>
Body: { "sessionId": "<uuid>" }
```

逻辑： 1. 验证 JWT → 获取 userId 2. 查询 `qr_login_sessions` 确认 session 存在且未过期 3. 调用 Supabase Admin API `generateLink` 生成一次性 token 4. 更新 `qr_login_sessions.status = 'approved'`，写入 `login_token` 5. Cloud 桌面通过 Realtime 收到更新 → 完成登录

10.4.2 4.2 `weather-proxy`

触发：Sync 应用调用

```
GET /functions/v1/weather-proxy?lat=30.1&lon=120.1
Authorization: Bearer <用户JWT>
```

逻辑： 1. 验证 JWT 2. 用 EdDSA 私钥生成和风天气 JWT（`QWEATHER_PROJECT_ID` / `QWEATHER_KID` / `QWEATHER_PRIVATE_KEY_B64`） 3. 调用 `{QWEATHER_API_HOST}/v7/weather/now?location=lon,lat` 4. 调用 `{QWEATHER_API_HOST}/v7/weather/3d?location=lon,lat` 5. 合并返回

10.4.3 4.3 `herb-proxy`

触发：Doctor 应用调用

```
GET /functions/v1/herb-proxy?p=/herb/search?q=人参
Authorization: Bearer <用户JWT>
```

逻辑： 1. 验证 JWT 2. 取 `p` 参数作为路径 3. `GET {HERB_API_BASE}{p}?key={HERB_API_KEY}` 4. 透传响应

10.4.4 4.4 `cloud-files`

触发：Cloud 应用调用（见第 7 章详细说明）

```
POST /functions/v1/cloud-files?action=list
Authorization: Bearer <用户JWT>
Body: { "prefix": "owners/{uid}/documents/" }
```

强制前缀 `owners/{uid}/`，防越权访问。

10.5 5. Storage Bucket

Bucket	说明	访问控制
`app-releases`	应用安装包	公开可读；写入需管理员权限
`cloud-user-files`	用户网盘文件	通过 `cloud-files` 函数操作（S3 签名）

10.6 6. Realtime 订阅表清单

表名	加入时机	使用方
`qr_login_sessions`	迁移 `20240101000001`	Talk / Cloud
`cnchess_presence`	迁移 `20260413000102`	CnChess
`chess_invites`	迁移 `20260413000102`	CnChess
`chess_games`	迁移 `20260413000102`	CnChess
`chess_moves`	迁移 `20260413000203`	CnChess
`friendships`	迁移 `20260415234000`	Talk

10.7 7. 迁移文件说明

文件	内容
`20240101000001_qr_login_sessions.sql`	扫码登录基础表 + Realtime
`20260413000102_cnchess_invite_game.sql`	象棋邀请/presence/游戏表 + Realtime
`20260413000203_cnchess_gameplay.sql`	走子记录表 + Realtime
`20260413000304_cnchess_judge_v2.sql`	棋规字段 + make_move_v2 初版
`20260414000100_cnchess_draw_offer.sql`	和棋提议字段 + RPC 更新
`20260414082906_remote_schema.sql`	全量 Schema 快照（友谊/消息/Sync指标/天气等）
`20260415120000_cnchess_professional_clock.sql`	专业棋钟逻辑改进
`20260415234000_add_friendships_to_realtime.sql`	friendships 加入 Realtime

下一章：跨模块核心流程

11 跨模块核心流程

> 本章梳理平台中涉及多个应用协作的关键业务流程，帮助理解模块间的数据依赖关系。

11.1 1. 统一账号注册与登录

涉及模块：全部应用（Sync / Talk / Doctor / CnChess / Music / Cloud）

所有应用共享同一套 Supabase Auth，用户在任一应用注册后，凭相同邮箱密码可登录所有应用。

```
用户输入邮箱 → AuthEmailRoute
      │
      ├── 已有账号：输入密码 → AuthPasswordRoute
      │       │
      │       ▼
      │   supabase.auth.signInWithPassword(email, password)
      │
      └── 新用户：填写密码注册 → signUp(email, password)
              │
              ▼
         auth.users 新增记录
              │
```

```
                ▼（触发器 on_auth_user_created）
        profiles 自动创建行（display_name 取自邮箱前缀）
                │
                ▼
        PKCE 会话建立 → 各应用 AuthSessionViewModel 感知
                │
                ▼
        isInitializing=false, isAuthenticated=true → 进入主界面
```

密码重置:

```
Auth 页面 → 点击"忘记密码" → signIn with PKCE link → 邮件
      │
收到邮件 → 点击链接 → 深链回调（如 talkapp://login#...）
      │
supabase.auth.exchangeCodeForSession() → 恢复会话
      │
UpdatePasswordDialog 弹出 → 输入新密码 → updateUser(password: xxx)
```

11.2 2. 扫码授权登录（手机→桌面）

涉及模块：Talk（手机扫码）← → Cloud Desktop（桌面展示 QR）

```
┌──────────────────────────────────────────────────────────────────┐
│                          时序图                                  │
│                                                                  │
│  Cloud Desktop          Supabase              Talk (手机)        │
│      │                      │                     │             │
│      │ INSERT qr_session    │                     │             │
│      │ status='pending'     │                     │             │
│      │──────────────────────►│                     │             │
│      │                      │                     │             │
│      │ 展示 QR（含sessionId）│                     │             │
│      │                      │                     │             │
│      │ Realtime SUBSCRIBE   │                     │             │
│      │ qr_login_sessions    │                     │             │
│      │──────────────────────►│                     │             │
│      │                      │                     │             │
│      │                      │◄────────────────────│             │
│      │                      │  扫码解析 sessionId  │             │
│      │                      │                     │             │
│      │                      │◄────────────────────│             │
│      │                      │  POST approve-login  │             │
│      │                      │  (JWT + sessionId)   │             │
│      │                      │                     │             │
│      │                      │ 验证 JWT             │             │
│      │                      │ generateLink(token)  │             │
│      │                      │ UPDATE session       │             │
│      │                      │ status='approved'    │             │
│      │                      │ login_token=xxx      │             │
│      │                      │                     │             │
```

```
│       │◄──────────────────────│                        │             │
│       │ Realtime UPDATE       │                        │             │
│       │ 收到 login_token      │                        │             │
│       │                       │                        │             │
│       │ exchangeToken(token)  │                        │             │
│       │──────────────────────►│                        │             │
│       │                       │                        │             │
│       │◄──────────────────────│                        │             │
│       │ 会话建立成功          │                        │             │
│       │ → 进入 ExplorerScreen │                        │             │
└──────────────────────────────────────────────────────────────────────┘
```

关键安全保障： - QR 二维码包含 `sessionId`，本身无任何凭证 - 只有携带有效 JWT 的手机端可调用 `approve-login` - `login_token` 是 Supabase 生成的一次性 token，不可复用 - session 有 `expires_at`，过期后 QR 自动失效

11.3 3. 好友系统（Talk ↔ CnChess 共享）

涉及模块：Talk（管理好友）、CnChess（好友对局）

`friendships` 表由 Talk 管理（搜索、申请、同意、拒绝），CnChess 直接读取同一张表展示好友列表并发起象棋邀请。

```
Talk：用户 A 添加用户 B 为好友（接受申请后）
      │
      ▼
数据库：friendships.status = 'accepted', room_id = <新房间>
      │
      │──────────────────────────────────────────────┐
      ▼                                                ▼
Talk：                                            CnChess：
好友出现在联系人列表                              好友出现在首页列表
可发起聊天（进入 room）                           可发起象棋邀请
```

好友关系的 CnChess 使用： - 首页 `FriendsScreen` 直接查询 `friendships`（accepted + 自身为一方） - `cnchess_presence` RLS：只有已接受好友的在线状态可见 - 邀请发送：目标方必须在好友列表中

11.4 4. 象棋邀请与对局建立

涉及模块：CnChess（全流程）+ 共享 `friendships`

```
用户 A 在好友列表点击「邀请对局」
      │
ChessInviteRepository.sendInvite(toUserId)
      │  INSERT chess_invites(from=A, to=B, status='pending')
      ▼
B 的 InviteListenerViewModel 收到 Realtime INSERT 事件
```

```
      │
IncomingInviteDialog 弹出（带接受/拒绝按钮）
      │
      ├── 拒绝 → UPDATE status='rejected' → A 收到通知
      │
      └── 接受 → acceptInvite(inviteId)
              │  服务端：INSERT chess_games(red=A, black=B)
              │  服务端：UPDATE chess_invites.game_id = newGameId,
status='accepted'
              ▼
          A 和 B 的 ActiveGameViewModel.checkOnce() 检测到进行中对局
              │
              ▼
          自动导航到 GameScreen(gameId)
              │
              ▼
          对局开始（棋盘初始化，红方先走）
```

11.5 5. 应用版本发布与更新

涉及模块：Admin Web（发布）← → 全部 Android 应用（检查）

```
┌──────────────────────────────────────────────────────────────┐
│                        发布侧（管理员）                        │
│                                                              │
│  Admin Web                                                   │
│  1. 填写版本信息 + 选择安装包                                  │
│  2. 上传 APK 到 Storage bucket: app-releases                 │
│  3. INSERT app_releases（含 download_url）                    │
│  4. 旧版本自动 enabled=false（触发器）                          │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ 实时（Realtime 或下次启动）
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                        消费侧（用户）                          │
│                                                              │
│  Android 应用启动                                             │
│  AppUpdateGate → 查询 app_releases                            │
│  WHERE package_name = '当前包名'                               │
│    AND enabled = true                                        │
│                                                              │
│  检查结果：                                                    │
│  ├── 无记录 / 版本相同 → 正常启动                               │
│  ├── 有新版本 + force=false → 可跳过更新弹窗                    │
│  └── 有新版本 + force=true → 不可跳过 → 下载 download_url → 安装 │
└──────────────────────────────────────────────────────────────┘
```

灰度发布：`is_rollout=true` 的版本只对特定用户可见（可配合服务端逻辑决定哪些 uid 命中灰度）。

11.6 6. 天气数据流（Sync → 和风 API）

涉及模块：Sync（消费）→ Edge Function（代理）→ 第三方（和风天气）

```
Sync 应用前台
      │
定位模块获取经纬度（Play Services Location）
      │
WeatherRepositoryImpl.fetchWeather(lat, lon)
      │  检查 Room 缓存是否过期
      │
      ├── 未过期 → 直接返回缓存
      │
      └── 已过期 → GET /functions/v1/weather-proxy?lat=xxx&lon=xxx
                   │  Header: Authorization: Bearer <JWT>
                   ▼
              weather-proxy 函数
                   │  EdDSA 签名 → 和风天气 API
                   ▼
              返回 JSON（now + 3d 预报）
                   │
                   ▼
              写入 Room WeatherSnapshot + 上传 weather_snapshots 表
                   │
                   ▼
              WeatherViewModel → UI 刷新天气卡片
```

11.7 7. 中医内容访问（Doctor → zyapi）

涉及模块：Doctor（消费）→ Edge Function（代理）→ 第三方（本草 API）

```
Doctor 应用搜索/浏览文章
      │
HerbRepository.searchHerbs(query)
      │  先查 Room 缓存
      │
      ├── 命中缓存 → 返回缓存数据（离线可用）
      │
      └── 未命中 → GET /functions/v1/herb-proxy?p=/herb/search?q=xxx
                   │  Header: Authorization: Bearer <JWT>
                   ▼
              herb-proxy 函数
                   │  附上 HERB_API_KEY → 转发到 HERB_API_BASE
                   ▼
              第三方本草 API 响应
                   │
                   ▼
              写入 Room 缓存
                   │
```

```
                ▼
          ViewModel → UI 展示文章列表
```

11.8 8. 跨模块依赖总结

```
┌──────────────────────────────────────────────────────────────────────┐
│                          数据依赖关系图                                │
│                                                                      │
│  Talk ──────────────► friendships ◄──────────── CnChess              │
│  (管理好友)              (共享表)              (读取好友)              │
│                                                                      │
│  Talk ──────────────► qr_login_sessions ◄─── Cloud Desktop           │
│  (批准登录)             (共享表)              (发起/等待)              │
│                                                                      │
│  Admin Web ─────────► app_releases ◄──────── 全部 Android 应用        │
│  (发布版本)              (共享表)              (检查更新)              │
│                                                                      │
│  所有应用 ──────────► profiles / auth.users                           │
│  (使用同一账号)          (共享认证体系)                                 │
│                                                                      │
│  Sync ──────────────► weather-proxy ──────► 和风天气 API              │
│  Doctor ────────────► herb-proxy ─────────► 本草 API                  │
│  Cloud ─────────────► cloud-files ────────► S3 存储                   │
│  Talk/Sync ─────────► approve-login ──────► qr_login_sessions         │
└──────────────────────────────────────────────────────────────────────┘
```

文档完毕。如需生成 PDF，请运行 build-docs.ps1。